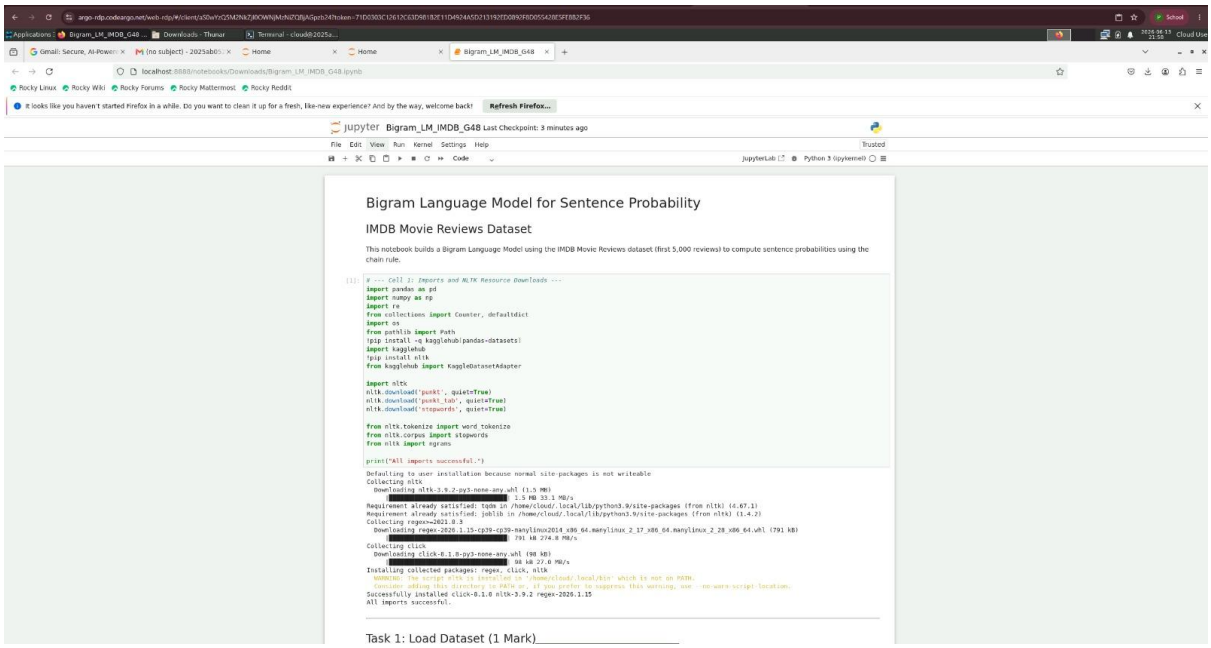
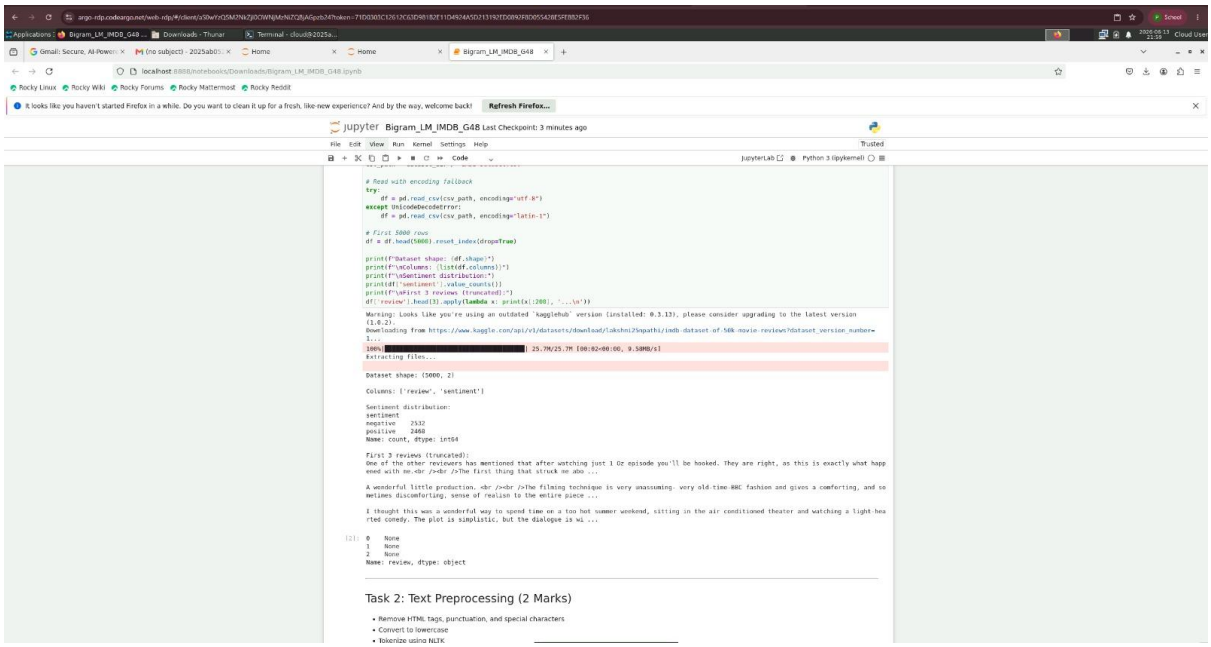
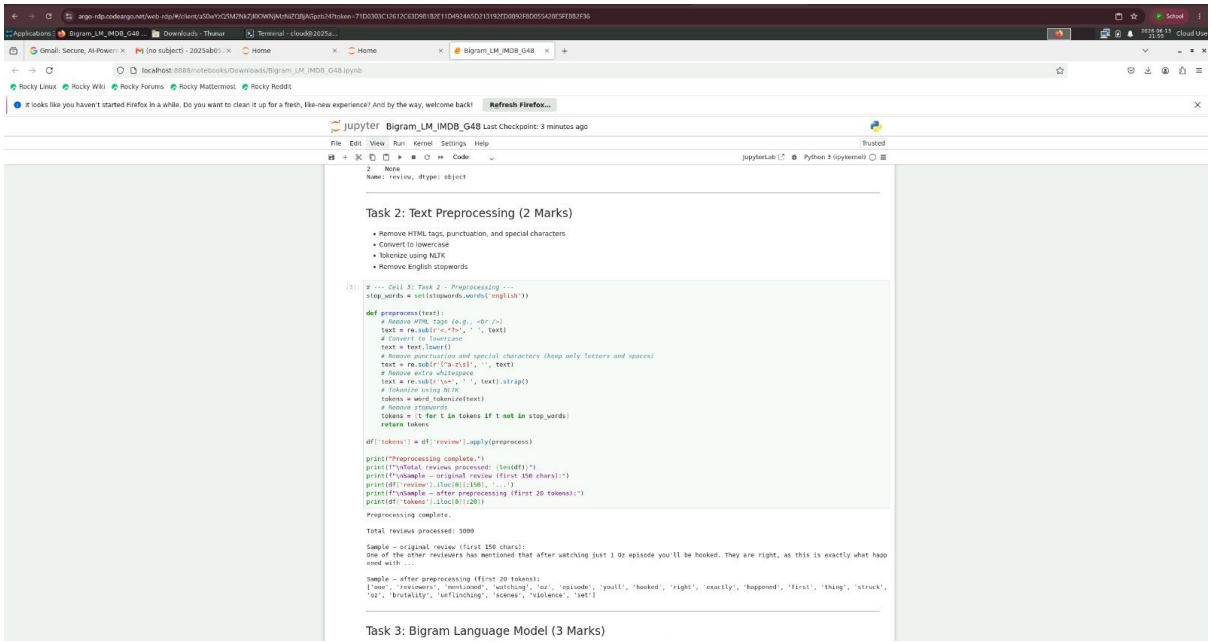
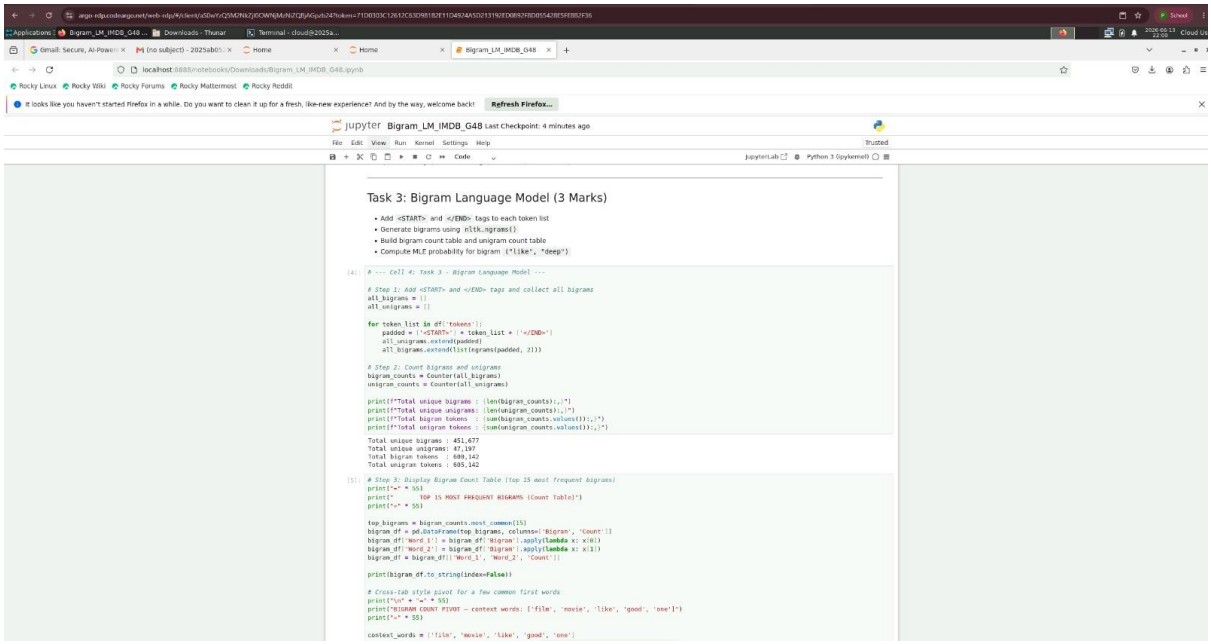

jupyter Bigram_LM_IMDB_G48 Last Checkpoint: 3 minutes ago
Bigram Language Model for Sentence Probability
IMDB Movie Reviews Dataset
This notebook builds a Bigram Language Model using the IMDB Movie Reviews dataset (first 5,000 reviews) to compute sentence probabilities using the chain rule.
# --- Cell 1: Imports and NLTK Resource Downloads ---
import pandas as pd
import numpy as np
import re
from collections import Counter, defaultdict
import os
from pathlib import Path
!pip install -q kagglehub[pandas-datasets]
import kagglehub
!pip install nltk
from kagglehub import KaggleDatasetAdapter
import nltk
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import ngrams
print("All imports successful.")
Defaulting to user installation because normal site-packages is not writeable
Collecting nltk
Downloading nltk-3.9.2-py3-none-any.whl (1.5 MB)
1.5 MB 33.1 MB/s
Requirement already satisfied: tqdm in /home/cloud/.local/lib/python3.9/site-packages (from nltk) (4.67.1)
Requirement already satisfied: joblib in /home/cloud/.local/lib/python3.9/site-packages (from nltk) (1.4.2)
Collecting regex>=2021.8.3
Downloading regex-2026.1.15-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (791 kB)
791 kB 274.8 MB/s
Collecting click
Downloading click-8.1.8-py3-none-any.whl (98 kB)
98 kB 27.6 MB/s
Installing collected packages: regex, click, nltk
WARNING: The script nltk is installed in '/home/cloud/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed click-8.1.8 nltk-3.9.2 regex-2026.1.15
All imports successful.
Task 1: Load Dataset (1 Mark)

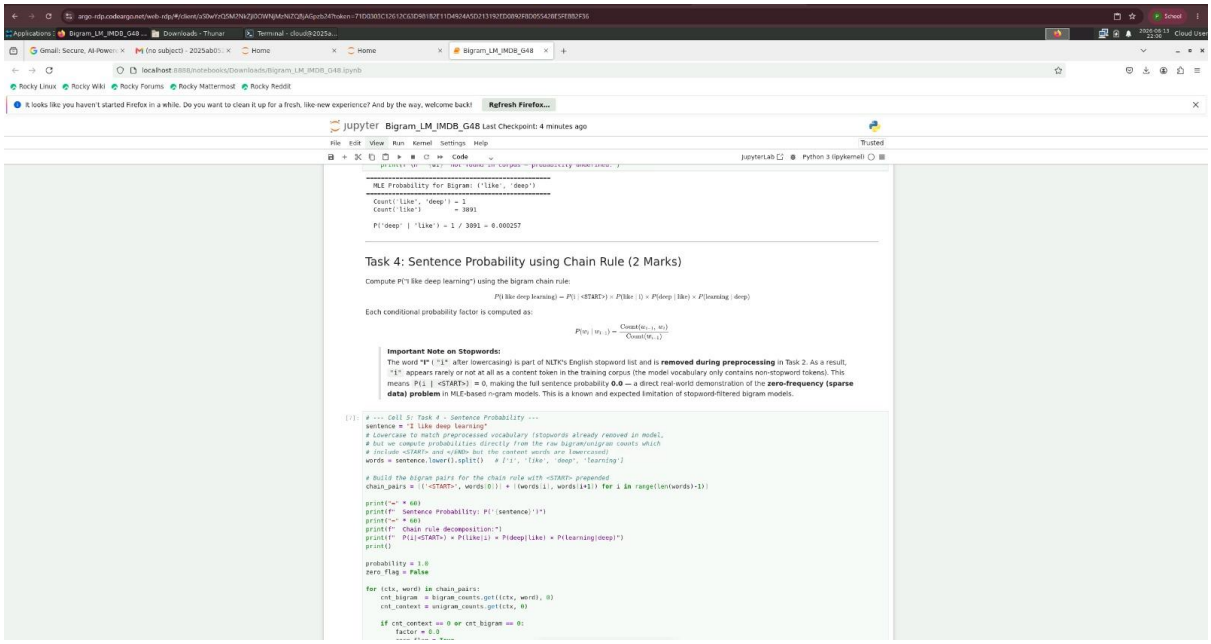

jupyter Bigram_LM_IMDB_G48 Last Checkpoint: 3 minutes ago
# Read with encoding fallback
try:
df = pd.read_csv(csv_path, encoding="utf-8")
except UnicodeDecodeError:
df = pd.read_csv(csv_path, encoding="latin-1")
# First 5000 rows
df = df.head(5000).reset_index(drop=True)
print(f"Dataset shape: {df.shape}")
print(f"\nColumns: {list(df.columns)}")
print(f"\nSentiment distribution:")
print(df['sentiment'].value_counts())
print(f"\nFirst 3 reviews (truncated):")
df['review'].head(3).apply(lambda x: print(x[:200], '...\n'))
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).
Downloading from https://www.kaggle.com/api/v1/datasets/download/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews?dataset_version_number=1...
100%
25.7M/25.7M [00:02<00:00, 9.58MB/s]
Extracting files...
Dataset shape: (5000, 2)
Columns: ['review', 'sentiment']
Sentiment distribution:
sentiment
negative 2532
positive 2468
Name: count, dtype: int64
First 3 reviews (truncated):
One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is exactly what happened with me.

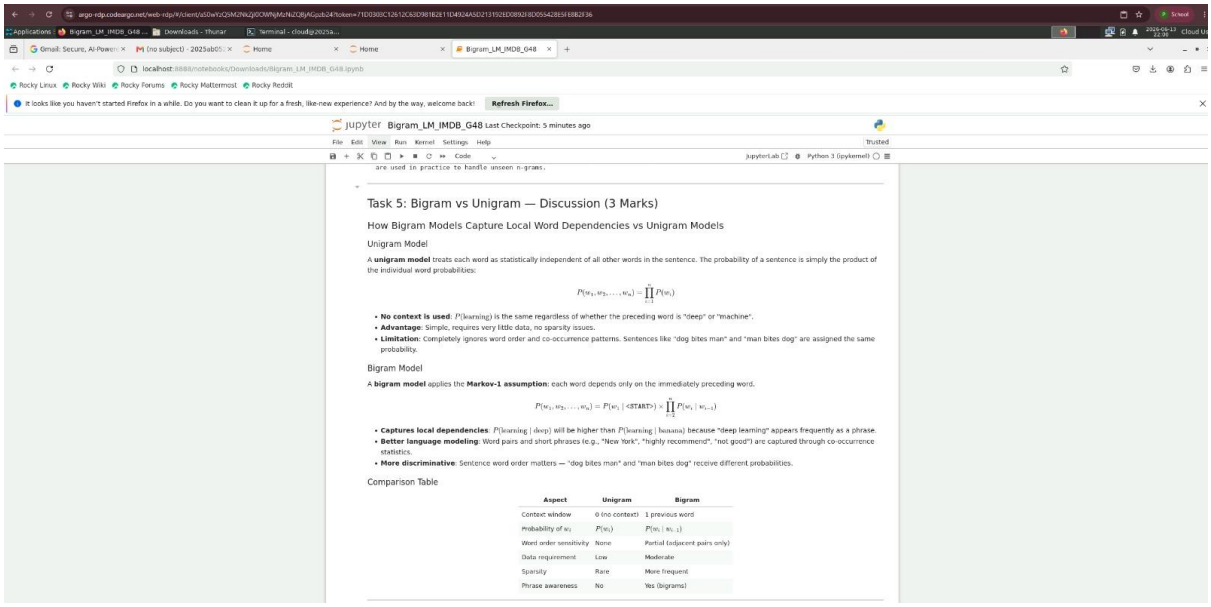
The first thing that struck me abo ...
A wonderful little production.

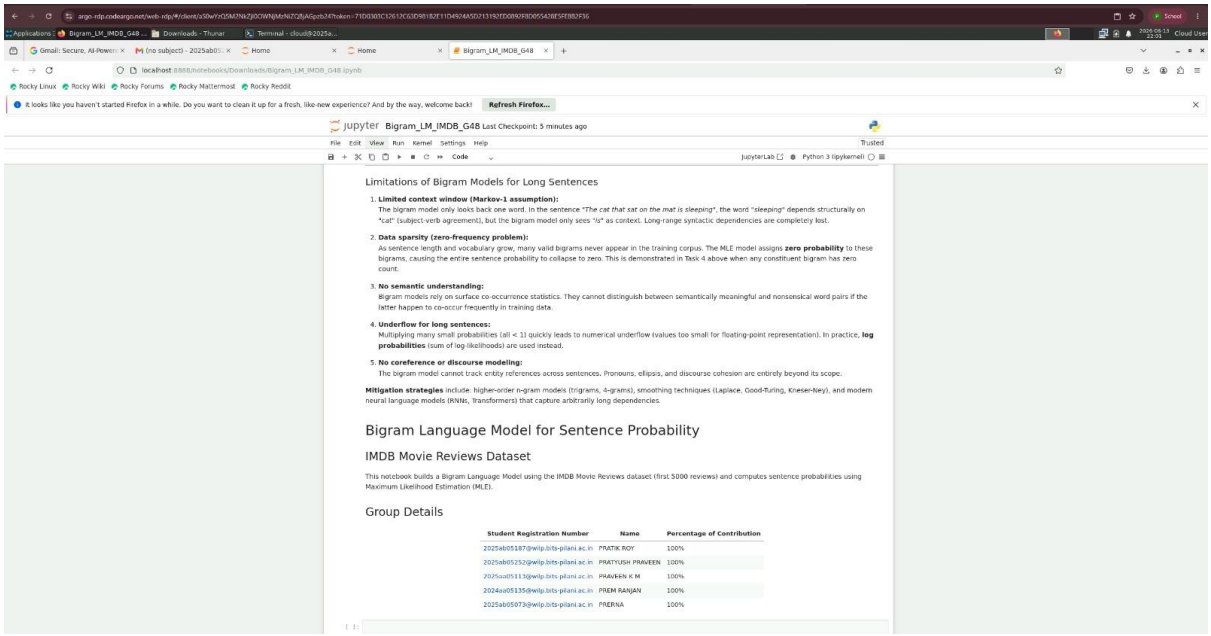
The filming technique is very unassuming. very old-time-BBC fashion and gives a comforting, and sometimes discomforting, sense of realism to the entire piece ...
I thought this was a wonderful way to spend time on a too hot summer weekend, sitting in the air conditioned theater and watching a light hearted comedy. The plot is simplistic, but the dialogue is wi ...
0 None
1 None
2 None
Name: review, dtype: object
Task 2: Text Preprocessing (2 Marks)
Remove HTML tags, punctuation, and special characters
Convert to lowercase
Tokenize using NLTK


jupyter Bigram_LM_IMDB_G48 Last Checkpoint: 3 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
2 None
Name: review, dtype: object
Task 2: Text Preprocessing (2 Marks)
• Remove HTML tags, punctuation, and special characters
• Convert to lowercase
• Tokenize using NLTK
• Remove English stopwords
[3]:
# --- Cell 3: Task 2 - Preprocessing ---
stop_words = set(stopwords.words('english'))
def preprocess(text):
# Remove HTML tags (e.g., <br />)
text = re.sub(r'<.*?>', ' ', text)
# Convert to lowercase
text = text.lower()
# Remove punctuation and special characters (keep only letters and spaces)
text = re.sub(r'[^a-z\s]', '', text)
# Remove extra whitespace
text = re.sub(r'\s+', ' ', text).strip()
# Tokenize using NLTK
tokens = word_tokenize(text)
# Remove stopwords
tokens = [t for t in tokens if t not in stop_words]
return tokens
df['tokens'] = df['review'].apply(preprocess)
print("Preprocessing complete.")
print(f"\nTotal reviews processed: {len(df)}")
print(f"\nSample – original review (first 150 chars):")
print(df['review'].iloc[0][:150], '...')
print(f"\nSample – after preprocessing (first 20 tokens):")
print(df['tokens'].iloc[0][:20])
Preprocessing complete.
Total reviews processed: 5000
Sample – original review (first 150 chars):
One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is exactly what happ
ened with ...
Sample – after preprocessing (first 20 tokens):
['one', 'reviewers', 'mentioned', 'watching', 'oz', 'episode', 'youll', 'hooked', 'right', 'exactly', 'happened', 'first', 'thing', 'struck',
'oz', 'brutality', 'unflinching', 'scenes', 'violence', 'set']
Task 3: Bigram Language Model (3 Marks)


jupyter Bigram_LM_IMDB_G48 Last Checkpoint: 4 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Task 3: Bigram Language Model (3 Marks)
• Add <START> and </END> tags to each token list
• Generate bigrams using nltk.ngrams()
• Build bigram count table and unigram count table
• Compute MLE probability for bigram ("like", "deep")
[4]:
# --- Cell 4: Task 3 - Bigram Language Model ---
# Step 1: Add <START> and </END> tags and collect all bigrams
all_bigrams = []
all_unigrams = []
for token_list in df['tokens']:
padded = ['<START>'] + token_list + ['</END>']
all_unigrams.extend(padded)
all_bigrams.extend(list(ngrams(padded, 2)))
# Step 2: Count bigrams and unigrams
bigram_counts = Counter(all_bigrams)
unigram_counts = Counter(all_unigrams)
print(f"Total unique bigrams : {len(bigram_counts):,}")
print(f"Total unique unigrams: {len(unigram_counts):,}")
print(f"Total bigram tokens : {sum(bigram_counts.values()):,}")
print(f"Total unigram tokens : {sum(unigram_counts.values()):,}")
Total unique bigrams : 451,677
Total unique unigrams: 47,197
Total bigram tokens : 689,142
Total unigram tokens : 695,142
[5]:
# Step 3: Display Bigram Count Table (top 15 most frequent bigrams)
print("=" * 55)
print(" TOP 15 MOST FREQUENT BIGRAMS (Count Table)")
print("=" * 55)
top_bigrams = bigram_counts.most_common(15)
bigram_df = pd.DataFrame(top_bigrams, columns=['Bigram', 'Count'])
bigram_df['Word_1'] = bigram_df['Bigram'].apply(lambda x: x[0])
bigram_df['Word_2'] = bigram_df['Bigram'].apply(lambda x: x[1])
bigram_df = bigram_df[['Word_1', 'Word_2', 'Count']]
print(bigram_df.to_string(index=False))
# Cross-tab style pivot for a few common first words
print("\n" + "=" * 55)
print("BIGRAM COUNT PIVOT – context words: ['film', 'movie', 'like', 'good', 'one']")
print("=" * 55)
context_words = ['film', 'movie', 'like', 'good', 'one']


jupyter Bigram_LM_IMDB_G48 Last Checkpoint: 4 minutes ago
MLE Probability for Bigram: ('like', 'deep')
Count('like', 'deep') = 1
Count('like') = 3891
P('deep' | 'like') = 1 / 3891 = 0.000257
Task 4: Sentence Probability using Chain Rule (2 Marks)
Compute P("I like deep learning") using the bigram chain rule:
Each conditional probability factor is computed as:
Important Note on Stopwords:
The word "I" ("i" after lowercasing) is part of NLTK's English stopword list and is removed during preprocessing in Task 2. As a result, "i" appears rarely or not at all as a content token in the training corpus (the model vocabulary only contains non-stopword tokens). This means P(i | <START>) = 0, making the full sentence probability 0.0 — a direct real-world demonstration of the zero-frequency (sparse data) problem in MLE-based n-gram models. This is a known and expected limitation of stopword-filtered bigram models.
# --- Cell 5: Task 4 - Sentence Probability ---
sentence = "I like deep learning"
# Lowercase to match preprocessed vocabulary (stopwords already removed in model,
# but we compute probabilities directly from the raw bigram/unigram counts which
# include <START> and </END> but the content words are lowercased)
words = sentence.lower().split() # ['i', 'like', 'deep', 'learning']
# Build the bigram pairs for the chain rule with <START> prepended
chain_pairs = [('<START>', words[0])] + [(words[i], words[i+1]) for i in range(len(words)-1)]
print("=" * 60)
print(f" Sentence Probability: P('{sentence}')")
print("=" * 60)
print(f" Chain rule decomposition:")
print(f" P(i|<START>) x P(like|i) x P(deep|like) x P(learning|deep)")
print()
probability = 1.0
zero_flag = False
for (ctx, word) in chain_pairs:
    cnt_bigram = bigram_counts.get((ctx, word), 0)
    cnt_context = unigram_counts.get(ctx, 0)
    if cnt_context == 0 or cnt_bigram == 0:
        factor = 0.0


jupyter Bigram_LM_IMDB_G48 Last Checkpoint: 5 minutes ago
are used in practice to handle unseen n-grams.
Task 5: Bigram vs Unigram — Discussion (3 Marks)
How Bigram Models Capture Local Word Dependencies vs Unigram Models
Unigram Model
A unigram model treats each word as statistically independent of all other words in the sentence. The probability of a sentence is simply the product of the individual word probabilities:
No context is used: P(learning) is the same regardless of whether the preceding word is "deep" or "machine".
Advantage: Simple, requires very little data, no sparsity issues.
Limitation: Completely ignores word order and co-occurrence patterns. Sentences like "dog bites man" and "man bites dog" are assigned the same probability.
Bigram Model
A bigram model applies the Markov-1 assumption: each word depends only on the immediately preceding word.
Captures local dependencies: P(learning | deep) will be higher than P(learning | banana) because "deep learning" appears frequently as a phrase.
Better language modeling: Word pairs and short phrases (e.g., "New York", "highly recommend", "not good") are captured through co-occurrence statistics.
More discriminative: Sentence word order matters — "dog bites man" and "man bites dog" receive different probabilities.
Comparison Table
Aspect
Unigram
Bigram
Context window
0 (no context)
1 previous word
Probability of w_i
$P(w_i)$
$P(w_i \mid w_{i-1})$
Word order sensitivity
None
Partial (adjacent pairs only)
Data requirement
Low
Moderate
Sparsity
Rare
More frequent
Phrase awareness
No
Yes (bigrams)


Bigram_LM_IMDB_G48 Last Checkpoint: 5 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Limitations of Bigram Models for Long Sentences
1. Limited context window (Markov-1 assumption):
The bigram model only looks back one word. In the sentence "The cat that sat on the mat is sleeping", the word "sleeping" depends structurally on "cat" (subject-verb agreement), but the bigram model only sees "is" as context. Long-range syntactic dependencies are completely lost.
2. Data sparsity (zero-frequency problem):
As sentence length and vocabulary grow, many valid bigrams never appear in the training corpus. The MLE model assigns zero probability to these bigrams, causing the entire sentence probability to collapse to zero. This is demonstrated in Task 4 above when any constituent bigram has zero count.
3. No semantic understanding:
Bigram models rely on surface co-occurrence statistics. They cannot distinguish between semantically meaningful and nonsensical word pairs if the latter happen to co-occur frequently in training data.
4. Underflow for long sentences:
Multiplying many small probabilities (all < 1) quickly leads to numerical underflow (values too small for floating-point representation). In practice, log probabilities (sum of log-likelihoods) are used instead.
5. No coreference or discourse modeling:
The bigram model cannot track entity references across sentences. Pronouns, ellipsis, and discourse cohesion are entirely beyond its scope.
Mitigation strategies include: higher-order n-gram models (trigrams, 4-grams), smoothing techniques (Laplace, Good-Turing, Kneser-Ney), and modern neural language models (RNNs, Transformers) that capture arbitrarily long dependencies.
Bigram Language Model for Sentence Probability
IMDB Movie Reviews Dataset
This notebook builds a Bigram Language Model using the IMDB Movie Reviews dataset (first 5000 reviews) and computes sentence probabilities using Maximum Likelihood Estimation (MLE).
Group Details
Student Registration Number
Name
Percentage of Contribution
2025ab05187@wilp.bits-pilani.ac.in
PRATIK ROY
100%
2025ab05252@wilp.bits-pilani.ac.in
PRATYUSH PRAVEEN
100%
2025aa05113@wilp.bits-pilani.ac.in
PRAVEEN K M
100%
2024aa05135@wilp.bits-pilani.ac.in
PREM RANJAN
100%
2025ab05073@wilp.bits-pilani.ac.in
PRERNA
100%

Bigram Language Model for Sentence Probability

IMDB Movie Reviews Dataset

This notebook builds a Bigram Language Model using the IMDB Movie Reviews dataset (first 5,000 reviews) to compute sentence probabilities using the chain rule.

Group ID - 48

Student Reg#	Name	% of Contribution
2025ab05187@wilp.bits-pilani.ac.in	PRATIK ROY	100%

2025ab05252@wilp.bits-pilani.ac.in | PRATYUSH PRAVEEN | 100% |
2025aa05113@wilp.bits-pilani.ac.in | PRAVEEN K M | 100% |
2024aa05135@wilp.bits-pilani.ac.in | PREM RANJAN | 100% |
2025ab05073@wilp.bits-pilani.ac.in | PRERNA | 100% |

In [1]:
```
import platform
print("System: ", platform.system())
print("Node Name: ", platform.node())
print("Release: ", platform.release())
print("Version: ", platform.version())
```

```
System:  Linux
Node Name:  2025aa05113
Release:  5.14.0-503.31.1.el9_5.x86_64
Version:  #1 SMP PREEMPT_DYNAMIC Tue Mar 11 16:53:43 UTC 2025
```

In [2]:
```
# --- Cell 1: Imports and NLTK Resource Downloads ---
import pandas as pd
import numpy as np
import re
from collections import Counter, defaultdict
import os
from pathlib import Path
import kagglehub
from kagglehub import KaggleDatasetAdapter

import nltk
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)

from nltk.tokenize import word_tokenize
```

```
from nltk.corpus import stopwords
from nltk import ngrams

print("All imports successful.")
```

```
All imports successful.
```

Task 1: Load Dataset (1 Mark)

Load the IMDB Dataset CSV file and use the first 5,000 reviews.

In [3]:
```
# --- Cell 2: Task 1 - Load Data ---

# Writable cache
cache_dir = Path.cwd() / ".kagglehub_cache"
cache_dir.mkdir(parents=True, exist_ok=True)

os.environ["KAGGLEHUB_CACHE"] = str(cache_dir)
dataset_handle = "lakshmi25npathi/imdb-dataset-of-50k-movie-reviews"

# Download dataset folder
dataset_dir = Path(kagglehub.dataset_download(dataset_handle))

# Build CSV path explicitly
csv_path = dataset_dir / "IMDB Dataset.csv"

# Read with encoding fallback
try:
    df = pd.read_csv(csv_path, encoding="utf-8")
except UnicodeDecodeError:
    df = pd.read_csv(csv_path, encoding="latin-1")

# First 5000 rows
df = df.head(5000).reset_index(drop=True)

print(f"Dataset shape: {df.shape}")
print(f"\nColumns: {list(df.columns)}")
print(f"\nSentiment distribution:")
print(df['sentiment'].value_counts())
print(f"\nFirst 3 reviews (truncated):")
df['review'].head(3).apply(lambda x: print(x[:200], '...\n'))
```

```
Warning: Looks like you're using an outdated `kagglehub` version (in
stalled: 0.3.13), please consider upgrading to the latest version (
1.0.2).
Downloading from https://www.kaggle.com/api/v1/datasets/download/lak
shmi25npathi/imdb-dataset-of-50k-movie-reviews?dataset_version_numbe
r=1...
100%|████████████████████████████████████████| 25.7M/25.7M [00:03<00:0
0, 8.62MB/s]
Extracting files...
```

```
Dataset shape: (5000, 2)

Columns: ['review', 'sentiment']

Sentiment distribution:
sentiment
negative    2532
positive    2468
Name: count, dtype: int64

First 3 reviews (truncated):
One of the other reviewers has mentioned that after watching just 1
Oz episode you'll be hooked. They are right, as this is exactly what
happened with me.<br /><br />The first thing that struck me abo ...

A wonderful little production. <br /><br />The filming technique is
very unassuming- very old-time-BBC fashion and gives a comforting, a
nd sometimes discomforting, sense of realism to the entire piece ...

I thought this was a wonderful way to spend time on a too hot summer
weekend, sitting in the air conditioned theater and watching a light
-hearted comedy. The plot is simplistic, but the dialogue is wi ...
```

Out[3]:
```
0    None
1    None
2    None
Name: review, dtype: object
```

Task 2: Text Preprocessing (2 Marks)

- Remove HTML tags, punctuation, and special characters
- Convert to lowercase
- Tokenize using NLTK
- Remove English stopwords

In [4]:
```
# --- Cell 3: Task 2 - Preprocessing ---
stop_words = set(stopwords.words('english'))

def preprocess(text):
    # Remove HTML tags (e.g., <br />)
    text = re.sub(r'<.*?>', ' ', text)
    # Convert to lowercase
    text = text.lower()
    # Remove punctuation and special characters (keep only letters
    text = re.sub(r'[^a-z\s]', '', text)
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    # Tokenize using NLTK
    tokens = word_tokenize(text)
    # Remove stopwords
    tokens = [t for t in tokens if t not in stop_words]
```

```
    return tokens

df['tokens'] = df['review'].apply(preprocess)

print("Preprocessing complete.")
print(f"\nTotal reviews processed: {len(df)}")
print(f"\nSample — original review (first 150 chars):")
print(df['review'].iloc[0][:150], '...')
print(f"\nSample — after preprocessing (first 20 tokens):")
print(df['tokens'].iloc[0][:20])
```

```
Preprocessing complete.

Total reviews processed: 5000

Sample — original review (first 150 chars):
One of the other reviewers has mentioned that after watching just 1
Oz episode you'll be hooked. They are right, as this is exactly what
happened with ...

Sample — after preprocessing (first 20 tokens):
['one', 'reviewers', 'mentioned', 'watching', 'oz', 'episode', 'youl
l', 'hooked', 'right', 'exactly', 'happened', 'first', 'thing', 'str
uck', 'oz', 'brutality', 'unflinching', 'scenes', 'violence', 'set']
```

Task 3: Bigram Language Model (3 Marks)

- Add `<START>` and `</END>` tags to each token list
- Generate bigrams using `nltk.ngrams()`
- Build bigram count table and unigram count table
- Compute MLE probability for bigram `("like", "deep")`

In [5]:
```
# --- Cell 4: Task 3 — Bigram Language Model ---

# Step 1: Add <START> and </END> tags and collect all bigrams
all_bigrams = []
all_unigrams = []

for token_list in df['tokens']:
    padded = ['<START>'] + token_list + ['</END>']
    all_unigrams.extend(padded)
    all_bigrams.extend(list(ngrams(padded, 2)))

# Step 2: Count bigrams and unigrams
bigram_counts = Counter(all_bigrams)
unigram_counts = Counter(all_unigrams)

print(f"Total unique bigrams : {len(bigram_counts):,}")
print(f"Total unique unigrams: {len(unigram_counts):,}")
print(f"Total bigram tokens  : {sum(bigram_counts.values()):,}")
print(f"Total unigram tokens : {sum(unigram_counts.values()):,}")
```

```
Total unique bigrams : 451,677
Total unique unigrams: 47,197
Total bigram tokens  : 600,142
Total unigram tokens : 605,142
```

In [6]:
```python
# Step 3: Display Bigram Count Table (top 15 most frequent bigrams)
print("=" * 55)
print("       TOP 15 MOST FREQUENT BIGRAMS (Count Table)")
print("=" * 55)

top_bigrams = bigram_counts.most_common(15)
bigram_df = pd.DataFrame(top_bigrams, columns=['Bigram', 'Count'])
bigram_df['Word_1'] = bigram_df['Bigram'].apply(lambda x: x[0])
bigram_df['Word_2'] = bigram_df['Bigram'].apply(lambda x: x[1])
bigram_df = bigram_df[['Word_1', 'Word_2', 'Count']]

print(bigram_df.to_string(index=False))

# Cross-tab style pivot for a few common first words
print("\n" + "=" * 55)
print("BIGRAM COUNT PIVOT — context words: ['film', 'movie', 'like'
print("=" * 55)

context_words = ['film', 'movie', 'like', 'good', 'one']
rows = []
for w1 in context_words:
    row = {'Context (w1)': w1}
    # find top 5 continuations for each context word
    continuations = {w2: cnt for (a, w2), cnt in bigram_counts.item
    top5 = sorted(continuations.items(), key=lambda x: -x[1])[:5]
    for rank, (w2, cnt) in enumerate(top5, 1):
        row[f'Top-{rank} word (count)'] = f"{w2} ({cnt})"
    rows.append(row)

pivot_df = pd.DataFrame(rows).fillna('-')
print(pivot_df.to_string(index=False))
```

```
============================================================
       TOP 15 MOST FREQUENT BIGRAMS (Count Table)
============================================================
 Word_1  Word_2  Count
   ever    seen    252
<START>   movie    234
    ive    seen    230
special effects    220
   dont    know    217
   even  though    204
  movie  </END>    178
    one    best    178
  looks    like    173
  waste    time    165
     im    sure    156
    see   movie    146
   good   movie    139
<START>    film    138
   much  better    133

============================================================
BIGRAM COUNT PIVOT — context words: ['film', 'movie', 'like', 'goo
d', 'one']
============================================================
Context (w1) Top-1 word (count) Top-2 word (count) Top-3 word (coun
t) Top-4 word (count) Top-5 word (count)
        film       </END> (126)         would (75)        really (7
3)           one (72)          made (70)
       movie       </END> (178)          one (105)         ever (10
2)        really (88)          like (86)
        like         movie (82)           one (61)          film (4
8)      watching (40)           see (34)
        good        movie (139)          film (75)           job (6
2)         thing (56)        acting (46)
         one         best (178)        worst (102)         thing (9
4)        </END> (69)      favorite (52)
```

In [7]:
```python
# Step 4: MLE Probability for bigram ("like", "deep")
w1, w2 = 'like', 'deep'

count_w1_w2 = bigram_counts.get((w1, w2), 0)
count_w1    = unigram_counts.get(w1, 0)

print("=" * 50)
print(f"  MLE Probability for Bigram: ('{w1}', '{w2}')")
print("=" * 50)
print(f"  Count('{w1}', '{w2}') = {count_w1_w2}")
print(f"  Count('{w1}')         = {count_w1}")

if count_w1 > 0:
    mle_prob = count_w1_w2 / count_w1
    print(f"\n  P('{w2}' | '{w1}') = {count_w1_w2} / {count_w1} = {
else:
    print(f"\n  '{w1}' not found in corpus — probability undefined.'
```

```
================================================
  MLE Probability for Bigram: ('like', 'deep')
================================================
  Count('like', 'deep') = 1
  Count('like')         = 3891

  P('deep' | 'like') = 1 / 3891 = 0.000257
```

Task 4: Sentence Probability using Chain Rule (2 Marks)

Compute P("I like deep learning") using the bigram chain rule:

$$P(\text{i like deep learning}) = P(\text{i} \mid \texttt{<START>}) \times P(\text{like} \mid \text{i}) \times P(\text{deep} \mid \text{like}) \times P$$

Each conditional probability factor is computed as:

$$P(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1},\ w_i)}{\text{Count}(w_{i-1})}$$

> **Important Note on Stopwords:**
> The word **"I"** (`"i"` after lowercasing) is part of NLTK's English stopword list and is **removed during preprocessing** in Task 2. As a result, `"i"` appears rarely or not at all as a content token in the training corpus (the model vocabulary only contains non-stopword tokens). This means `P(i | <START>)` = 0, making the full sentence probability **0.0** — a direct real-world demonstration of the **zero-frequency (sparse data) problem** in MLE-based n-gram models. This is a known and expected limitation of stopword-filtered bigram models.

In [8]:
```
# --- Cell 5: Task 4 - Sentence Probability ---
sentence = "I like deep learning"
# Lowercase to match preprocessed vocabulary (stopwords already rem
# but we compute probabilities directly from the raw bigram/unigram
# include <START> and </END> but the content words are lowercased)
words = sentence.lower().split()   # ['i', 'like', 'deep', 'learnin

# Build the bigram pairs for the chain rule with <START> prepended
chain_pairs = [('<START>', words[0])] + [(words[i], words[i+1]) for

print("=" * 60)
print(f"  Sentence Probability: P('{sentence}')")
print("=" * 60)
print(f"  Chain rule decomposition:")
print(f"  P(i|<START>) × P(like|i) × P(deep|like) × P(learning|deep
print()
```

```
probability = 1.0
zero_flag = False

for (ctx, word) in chain_pairs:
    cnt_bigram  = bigram_counts.get((ctx, word), 0)
    cnt_context = unigram_counts.get(ctx, 0)

    if cnt_context == 0 or cnt_bigram == 0:
        factor = 0.0
        zero_flag = True
        print(f"  P('{word}' | '{ctx}') = {cnt_bigram} / {cnt_contex
    else:
        factor = cnt_bigram / cnt_context
        print(f"  P('{word}' | '{ctx}') = {cnt_bigram} / {cnt_contex

    probability *= factor

print()
print("=" * 60)
print(f"  Final Sentence Probability = {probability:.10e}")
print("=" * 60)

if zero_flag:
    print()
    print("  NOTE: One or more bigrams were not seen in the corpus,"
    print("  resulting in a zero probability. This is a known limita
    print("  of MLE-based bigram models (the 'zero-frequency problem
    print("  Smoothing techniques (e.g., Laplace/Add-1, Kneser-Ney)"
    print("  are used in practice to handle unseen n-grams.")
```

```
============================================================
  Sentence Probability: P('I like deep learning')
============================================================
  Chain rule decomposition:
  P(i|<START>) × P(like|i) × P(deep|like) × P(learning|deep)

  P('i' | '<START>') = 0 / 5000 = 0  ← ZERO (unseen bigram)
  P('like' | 'i') = 0 / 0 = 0  ← ZERO (unseen bigram)
  P('deep' | 'like') = 1 / 3891 = 0.00025700
  P('learning' | 'deep') = 0 / 140 = 0  ← ZERO (unseen bigram)

============================================================
  Final Sentence Probability = 0.0000000000e+00
============================================================

  NOTE: One or more bigrams were not seen in the corpus,
  resulting in a zero probability. This is a known limitation
  of MLE-based bigram models (the 'zero-frequency problem').
  Smoothing techniques (e.g., Laplace/Add-1, Kneser-Ney)
  are used in practice to handle unseen n-grams.
```

Task 5: Bigram vs Unigram — Discussion (3 Marks)

How Bigram Models Capture Local Word Dependencies vs Unigram Models

Unigram Model

A **unigram model** treats each word as statistically independent of all other words in the sentence. The probability of a sentence is simply the product of the individual word probabilities:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i)$$

- **No context is used**: $P(\text{learning})$ is the same regardless of whether the preceding word is "deep" or "machine".
- **Advantage**: Simple, requires very little data, no sparsity issues.
- **Limitation**: Completely ignores word order and co-occurrence patterns. Sentences like "dog bites man" and "man bites dog" are assigned the same probability.

Bigram Model

A **bigram model** applies the **Markov-1 assumption**: each word depends only on the immediately preceding word.

$$P(w_1, w_2, \ldots, w_n) = P(w_1 \mid \texttt{<START>}) \times \prod_{i=2}^{n} P(w_i \mid w_{i-1})$$

- **Captures local dependencies**: $P(\text{learning} \mid \text{deep})$ will be higher than $P(\text{learning} \mid \text{banana})$ because "deep learning" appears frequently as a phrase.
- **Better language modeling**: Word pairs and short phrases (e.g., "New York", "highly recommend", "not good") are captured through co-occurrence statistics.
- **More discriminative**: Sentence word order matters — "dog bites man" and "man bites dog" receive different probabilities.

Comparison Table

Aspect	Unigram	Bigram
Context window	0 (no context)	1 previous word
Probability of w_i	$P(w_i)$	$P(w_i \mid w_{i-1})$
Word order sensitivity	None	Partial (adjacent pairs only)
Data requirement	Low	Moderate
Sparsity	Rare	More frequent

Phrase awareness	No	Yes (bigrams)

Limitations of Bigram Models for Long Sentences

1. **Limited context window (Markov-1 assumption):**
   The bigram model only looks back one word. In the sentence *"The cat that sat on the mat is sleeping"*, the word *"sleeping"* depends structurally on *"cat"* (subject-verb agreement), but the bigram model only sees *"is"* as context. Long-range syntactic dependencies are completely lost.

2. **Data sparsity (zero-frequency problem):**
   As sentence length and vocabulary grow, many valid bigrams never appear in the training corpus. The MLE model assigns **zero probability** to these bigrams, causing the entire sentence probability to collapse to zero. This is demonstrated in Task 4 above when any constituent bigram has zero count.

3. **No semantic understanding:**
   Bigram models rely on surface co-occurrence statistics. They cannot distinguish between semantically meaningful and nonsensical word pairs if the latter happen to co-occur frequently in training data.

4. **Underflow for long sentences:**
   Multiplying many small probabilities (all < 1) quickly leads to numerical underflow (values too small for floating-point representation). In practice, **log probabilities** (sum of log-likelihoods) are used instead.

5. **No coreference or discourse modeling:**
   The bigram model cannot track entity references across sentences. Pronouns, ellipsis, and discourse cohesion are entirely beyond its scope.

Mitigation strategies include: higher-order n-gram models (trigrams, 4-grams), smoothing techniques (Laplace, Good-Turing, Kneser-Ney), and modern neural language models (RNNs, Transformers) that capture arbitrarily long dependencies.